

URL Shortener Service - Proje Sunumu

Lütfen Markdown sunum araçları (Marp vb.) kullanarak PDF'e çevirin

Slayt 1: Kapak

URL Shortener Service

Bulut Mimarilerinde Test Mühendisliği (MTH2526-B25) Dönem Projesi

Ekip Üyeleri: [İsim 1], [İsim 2], [İsim 3]

Danışman: [Hoca Adı]

Slayt 2: Proje Özeti

Problem: Uzun ve karmaşık URL'lerin paylaşım zorluğu, tıklama verisinin takip edilememesi.

Çözüm: Mikroservis tabanlı, bulut teknolojileri ile izlenebilir bir link kısaltma servisi.

Temel Özellikler:

- Orijinal URL'yi `http://host/abc123` formuna çevirme
- Yönlendirme (HTTP 301) ve Tıklama Analizi
- API, Swagger Docs ve E2E UI
- Docker, K8s, Prometheus ve Grafana Entegrasyonu

Slayt 3: Mimari ve Teknoloji Yığını

Backend: FastAPI (Python 3.11), SQLAlchemy, Pydantic

Veritabanı: SQLite (Local) / PostgreSQL (Testcontainers)

Kapsayıcı: Docker (Multi-stage build)

Orkestrasyon: Kubernetes (Minikube)

Monitoring: Prometheus, Grafana

CI/CD: GitHub Actions

Bulut: LocalStack (S3 Simülasyonu)

Slayt 4: Test Stratejisi

- %70+ Pytest Code Coverage
- Birim (Unit) Testler:** İş mantığı, FactoryBoy & Faker veri üretimi
- Entegrasyon Testleri:** Testcontainers ile PostgreSQL testleri
- API Testleri (Postman/Newman):** 8 Senaryo, dinamik değişken kullanımı
- E2E Testleri:** Playwright ile UI etkileşimi (5 Senaryo)

Slayt 5: CI/CD Süreci

Tetikleyici: Push ve Pull Request'lerde otomatik çalışma

- Lint (Flake8 ile kod standartları)
- Test (Pytest ve Coverage kontrolü)
- Postman (API Entegrasyonu)
- Docker (İmaj Derleme ve Güvenlik)

5. Smoke Test (Canlı ortam simulasyon testi)

Slayt 6: Performans Testi

Araç: k6

Senaryo: 100 eşzamanlı kullanıcı (Isınma -> Yükleme -> Soğuma)

Ölçümler:

- %40 Shorten, %30 Redirect, %30 Stats/List dağılımı
 - Hedef p95 gecikme < 500ms
 - Başarı ve hata oranlarının Grafana üzerinde izlenmesi
-

Slayt 7: Monitoring ve Metrikler

Prometheus & Grafana:

- 6 Panelli özel Dashboard
 - Aktif URL sayısı
 - Saniyedeki istek (RPS) ve 404 Hata oranı
 - İstek Gecikmeleri (p50, p95, p99 Latency)
- (Burada Grafana ekran görüntüsü gösterilecek)
-

Slayt 8: Zorluklar & Kazanımlar

Zorluklar:

- Veritabanı izolasyon sorunları (Testler arası kirlilik)
 - Windows üzerinde bağımlılık derleme hataları (Pydantic, Psycopg2)
 - **Çözümler & Kazanımlar:**
 - `conftest.py` ve `yield` ile test izolasyonu yapmayı öğrendik.
 - Docker Multi-stage kullanarak her ortamda tutarlı çalışan yapılar kurduk.
 - Eksiksiz bir QA (Kalite Güvence) ve DevOps pipeline süreci inşa ettik.
-

Slayt 9: Teşekkürler ve Q&A

Projemizi dinlediğiniz için teşekkür ederiz. **Canlı Demo'ya geçiş yapıyoruz.** (Sorularınız?)